

weatherisk: A Python Package for Climate Risk Analysis via Max-Stable Process Clustering

Widad Kchikach^{a,*}

^a*Alfred Wegener Institute for Polar and Marine Research, Bremerhaven, Germany*

Abstract

We present **weatherisk**, an open-source Python package for quantifying regional climate risk through spatial extreme-value modelling. The package implements non-stationary max-stable processes on regular grids, estimates local anisotropy parameters via pairwise composite likelihood, and clusters grid cells by ellipse-shape dissimilarity to identify regions of homogeneous extremal dependence. Tail-risk metrics (Value at Risk and Expected Shortfall on the Fréchet scale) are computed per cluster, enabling spatially explicit hazard assessment for extremes such as heatwaves and heavy precipitation. Using 20 years of NOAA CPC daily precipitation data, we demonstrate a complete pipeline that produces filled-region cluster maps and per-cluster risk maps. An optional GDP exposure layer (Kummu et al., 2018) enables cell-level risk weighting, combining statistical tail-risk intensity with economic exposure. **weatherisk** consolidates a previously fragmented R/Shell/Python workflow into a single, tested, pip-installable library with a command-line interface, supporting both laptop-scale validation on synthetic data and analysis of global climate reanalysis products.

Keywords: climate risk, extreme-value theory, max-stable process, spatial clustering, anisotropy, Python

1. Motivation and significance

Extreme climate events — heatwaves, heavy precipitation — exhibit spatial dependence that must be modelled to quantify regional risk. Two nearby weather stations are likely to experience a heatwave simultaneously; ignoring this dependence leads to underestimation of aggregate losses [1]. Max-stable processes provide a principled framework for modelling such spatial extremes, allowing the joint tail behaviour of a spatial field to be characterised by a small number of parameters [2, 3].

A key challenge is *clustering*: partitioning a spatial domain into regions of homogeneous extremal dependence, so that within each region a single set of dependence parameters adequately describes the tail structure. Previous work by the authors [8] developed an R-based pipeline that (i) simulates non-stationary max-stable processes, (ii) estimates local parameters via pairwise

*Corresponding author

Email address: widad.kchikach@awi.de (Widad Kchikach)

Table 1: Code metadata.

Metadata description	Value
Current code version	0.1.0
Permanent link to code/repository	https://github.com/xxx/weatherisk
Permanent link to Reproducible Capsule	(to be added)
Legal Code License	MIT
Code versioning system used	git
Software code languages, tools, and services used	Python 3
Compilation requirements, operating environments & dependencies	Python ≥ 3.10 ; NumPy ≥ 1.24 ; SciPy ≥ 1.10 ; pandas ≥ 2.0 ; matplotlib ≥ 3.7 ; Click ≥ 8.0 ; xarray ≥ 2023.1 ; netCDF4 ≥ 1.6 ; scikit-learn ≥ 1.3 ; Cartopy ≥ 0.21 ; h5py ≥ 3.8
If available, link to developer documentation/manual	https://github.com/xxx/weatherisk
Support email for questions	widad.kchikach@awi.de

composite likelihood, (iii) constructs an ellipse-shape dissimilarity matrix, and (iv) applies hierarchical clustering. However, the original implementation is split across R scripts, SLURM shell orchestration, making reproducibility and extension limited.

weatherisk addresses this gap by consolidating the entire workflow into a single, well-tested Python package. It extends the original method with:

- extreme-value analysis (GEV fitting, block-maxima extraction, Fréchet transform);
- tail-risk metrics (VaR, ES) per cluster;
- a real-data pipeline for NOAA CPC daily climate data (precipitation and temperature), producing filled-region Cartopy maps;
- an optional GDP exposure layer for cell-level risk weighting;
- scalable estimation and clustering strategies for global grids; and
- a command-line interface with four pipeline flows.

No existing Python package provides this combination of non-stationary max-stable simulation, local composite-likelihood estimation, ellipse-based clustering, and per-cluster risk metrics. While **scikit-extremes** and **pyextremes** focus on univariate extreme-value analysis, and **SpatialExtremes** (R) implements max-stable simulation, none offer the integrated estimation-clustering-risk pipeline that **weatherisk** provides.

2. Software description

2.1. Software architecture

weatherisk is organised as a pip-installable Python package (`pyproject.toml`; PEP 621) with 18 modules, a CLI entry point, and 105 automated tests (including 50 cross-validation and R-parity tests against the original R code; see Section 4).

Figure 1 shows the package structure and data flow.

The modules are grouped into four tiers:

1. **Core mathematics** (no inter-module dependencies): `grid`, `covariance`, `parameters`, `extremes`, `risk`, `netcdf`, `io`.
2. **Simulation and estimation**: `simulation` (depends on `grid`, `covariance`), `density` (depends on `covariance`), `risk_pipeline` (standalone post-processing).
3. **Clustering and smoothing**: `estimation`, `clustering`, `scalable`.
4. **Orchestration**: `plotting`, `pipeline`, `cli`.

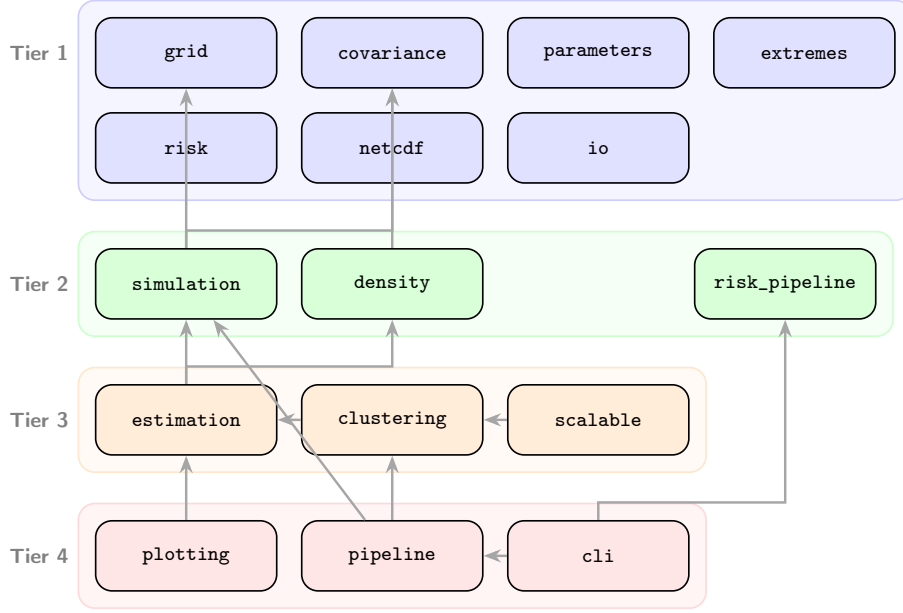


Figure 1: Module dependency structure of `weatherisk`. Tier 1 modules have no inter-package dependencies; higher tiers depend only on modules in the same or lower tiers. Arrows indicate import dependencies.

2.2. Software functionalities

2.2.1. Covariance model

The stationary anisotropic covariance function is

$$C(\mathbf{h}) = \exp(-\|\Sigma^{-1}\mathbf{h}\|^\alpha), \quad (1)$$

where $\mathbf{h} = (x, y)^T$ is the spatial lag, $\alpha \in (0, 2]$ is the smoothness exponent, and the precision matrix

$$\Sigma^{-1} = R(\gamma) \begin{pmatrix} 1/a & 0 \\ 0 & 1/(a+b) \end{pmatrix} R(\gamma)^T \quad (2)$$

encodes ellipse semi-axes $a > 0$, $a + b$ (semi-major), and rotation angle γ .

The non-stationary variant blends two anisotropy matrices M_1, M_2 (the inverse covariance matrices at locations 1 and 2) through the harmonic mean:

$$\Omega = \left(\frac{M_1^{-1} + M_2^{-1}}{2} \right)^{-1}, \quad (3)$$

$$C_{\text{ns}}(\mathbf{h}) = \min\left(1, \sqrt{\det(\Omega)} a_1(a_1 + b_1) a_2(a_2 + b_2) \exp(-\sqrt{\mathbf{h}^T \Omega \mathbf{h}}^\alpha)\right). \quad (4)$$

When both parameter sets are identical, $\Omega = M_1$ and the prefactor reduces to unity, recovering the stationary case.

2.2.2. Extremal coefficient

The extremal coefficient $\theta \in [1, 2]$ of the extremal- t model links pairwise dependence to correlation ρ via

$$\theta(\rho) = 2 T_{\nu+1} \left(\sqrt{\frac{(\nu+1)(1-\rho)}{1+\rho}} \right), \quad (5)$$

where $T_{\nu+1}$ is the CDF of the Student- t distribution with $\nu + 1$ degrees of freedom. At $\rho = 0$ with finite ν , $\theta < 2$ (residual tail dependence); complete independence ($\theta = 2$) is reached only as $\nu \rightarrow \infty$.

2.2.3. Max-stable process simulation

Simulation follows the spectral representation of [4]: a Poisson point process on $(0, \infty)$ with intensity $\zeta^{-2} d\zeta$ generates arrival times; for each arrival, a Gaussian spectral function is drawn from the multivariate normal with covariance matrix obtained via Cholesky decomposition of the grid covariance. The pointwise maximum over all spectral functions, scaled by the Poisson arrival, yields the max-stable realisation. Margins are Student- t with ν degrees of freedom, normalised to unit Fréchet via the probability integral transform.

2.2.4. Pairwise composite likelihood estimation

Local parameters (a, b, γ) at each grid cell are estimated by maximising the pairwise composite log-likelihood [3]:

$$\ell(\theta) = \sum_{i < j} \log f(z_i, z_j; \theta), \quad (6)$$

where the sum runs over all pairs within a spatial neighbourhood and f is the bivariate density implied by the covariance model. Optimisation uses Latin Hypercube initialisation followed by L-BFGS-B [5].

2.2.5. Clustering

Two clustering methods are implemented:

1. **Ellipse dissimilarity:** For each pair of grid cells, the Jaccard-like overlap between estimated anisotropy ellipses is computed and scaled to a dissimilarity in $[0, 100]$. Hierarchical agglomerative clustering (average linkage) is applied, and k is determined by a threshold on merge heights.
2. **Saunders method:** Rank-based madogram estimation of pairwise extremal coefficients yields an alternative dissimilarity matrix [6, 7].

2.2.6. Risk metrics

Given cluster assignments, **weatherisk** computes per-cluster tail-risk metrics on the Fréchet scale. For cluster A_k , the loss series is the spatial block maximum $L_t = \max_{s \in A_k} Z_t(s)$, where $Z_t(s)$ are the Fréchet-transformed annual maxima. Then:

$$\text{VaR}_p = \inf\{x : P(L \leq x) \geq p\}, \quad (7)$$

$$\text{ES}_p = E[L \mid L > \text{VaR}_p]. \quad (8)$$

These are hazard-intensity metrics: they quantify how severe the joint extreme events are within each cluster on a standardised scale, rather than in physical units. The implementation validates $\text{ES}_p \geq \text{VaR}_p$ by construction.

2.2.7. GDP exposure weighting

To connect statistical tail-risk to economic impact, **weatherisk** optionally incorporates gridded GDP data from Kummur et al. [9]. The GDP PPP grid (5 arc-min, constant 2011 international USD) is aggregated onto the coarser pipeline grid by summing fine-resolution cells within each coarse cell boundary. The cell-level risk is then defined as

$$\text{Risk}(s) = \text{ES}_{95}^{k(s)} \times \text{GDP}(s), \quad (9)$$

where $k(s)$ is the cluster to which cell s belongs. This avoids cluster-size bias: a city cell with high GDP within the same cluster as a desert cell appropriately receives a higher risk value despite sharing the same hazard intensity.

2.2.8. Scalable clustering for global grids

For grids exceeding $\sim 16\,000$ cells, the full dissimilarity matrix is prohibitively large. **weatherisk** provides a coarse-grid proxy strategy, illustrated in Figure 2.

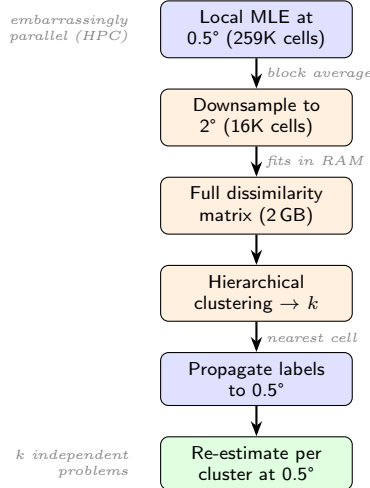


Figure 2: Coarse-grid proxy strategy for scalable clustering. Local estimation is embarrassingly parallel at full resolution; clustering operates on a downsampled grid that fits in memory; labels are propagated back and refined.

Intermediate results are checkpointed via `save_chunk/load_chunk`, enabling SLURM job-array parallelism on HPC systems.

2.3. Pipeline flows

The CLI exposes four pipeline flows, illustrated in Figure 3.

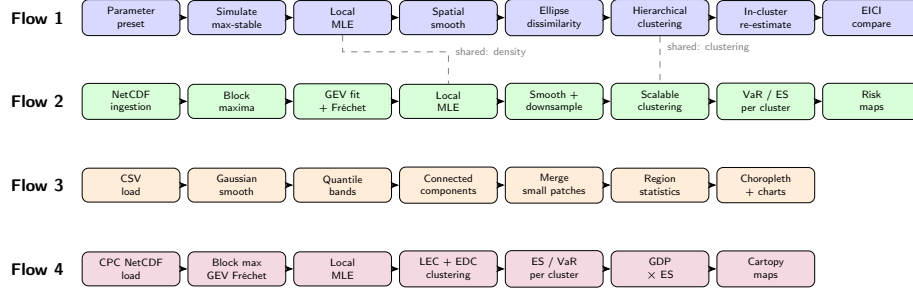


Figure 3: The four pipeline flows of **weatherisk**. Flow 1 validates the method on synthetic data. Flow 2 applies the full method to real climate observations with risk metrics. Flow 3 provides lightweight post-processing of pre-computed risk maps. Flow 4 runs the complete LEC/EDC pipeline on NOAA CPC data with optional GDP exposure weighting and generates filled-region Cartopy maps. Dashed lines indicate shared modules between flows.

*Flow 1: Method validation (**weatherisk validate**)*.. Reproduces the clustering method on synthetic data: parameters → simulate → local estimation → smooth → ellipse dissimilarity → hierarchical clustering → in-cluster re-estimation. Grid sizes of 10–51 points per axis run in minutes on a laptop.

```
$ weatherisk validate --params stripes \
  --resolution 10 --n-sim 20 \
  --seed 42
```

Listing 1: Running the validation flow.

*Flow 2: Real-data risk analysis (**weatherisk risk**)*.. Applies the method to observed climate data (CPC, ERA5): NetCDF ingestion → block maxima → GEV fit → Fréchet transform → local MLE → smooth → scalable clustering → VaR/ES per cluster → plots.

```
$ weatherisk risk \
  --netcdf \
  data/cpc_tmax.nc \
  --hazard heat \
  -k 25 --workers 8
```

Listing 2: Running the risk analysis flow.

*Flow 3: Risk pipeline.. **weatherisk riskpipeline***—lightweight post-processing of pre-computed risk maps: CSV → Gaussian smoothing → quantile banding → connected-component labelling → merge small patches → per-region statistics.

```
$ weatherisk risk-pipeline \
  --csv data/risk_map_grid.csv \
  --bands 6
```

Listing 3: Running the risk pipeline.

Flow 4: CPC maps (weatherisk maps).. The primary pipeline for reproducible analysis of NOAA CPC daily climate data. Loads 20 years of NetCDF files, computes annual block maxima, fits GEV marginals, transforms to unit Fréchet, performs local MLE of anisotropy parameters (a, b, γ) , runs both LEC and EDC clustering, and outputs filled-region Cartopy maps. When a GDP path is provided, computes cell-level exposure-weighted risk (Equation 9).

```
$ weatherisk maps \
  --variable precip \
  --gdp-path data/gdp/GDP_PPP.nc
```

Listing 4: Running the CPC maps pipeline with GDP exposure.

3. Illustrative examples

Figure 4 illustrates the mathematical transformation chain from raw observations to risk metrics.

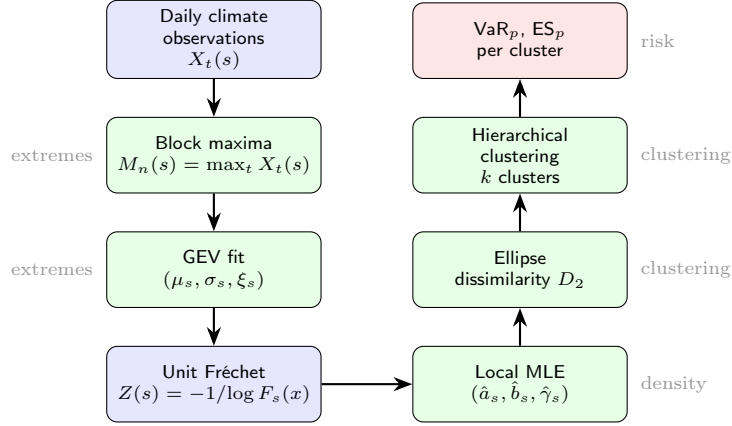


Figure 4: Mathematical transformation chain in Flow 2. Left column: marginal transformations from raw observations to unit Fréchet margins. Right column: dependence estimation, clustering, and risk quantification. Labels indicate the **weatherisk** module responsible for each step.

3.1. Synthetic validation

We demonstrate Flow 1 on the “stripes” parameter preset, where the semi-major axis b varies linearly with the x -coordinate, creating stripe-like anisotropy regions that the clustering method should recover.

```
from weatherisk.parameters import get_preset
from weatherisk.pipeline import run_pipeline
from weatherisk.plotting import plot_cluster_map

p = get_preset("stripes")
result = run_pipeline(
    resolution=10, n_sim=20,
    df=p.df, alpha=p.alpha, seed=42,
)
fig = plot_cluster_map(
    result["clusters"], result["grid"]
```

```
)
fig.savefig("clusters.pdf")
```

Listing 5: Python API usage for synthetic validation.

The pipeline produces cluster assignments that align with the known ground-truth anisotropy boundaries. On a standard laptop (Apple M2, 16 GB RAM), the 10×10 grid with 20 simulations completes in under 10 seconds.

Figure 5 illustrates the two clustering approaches. Placeholder panels will be replaced by generated output plots (see Section ??).

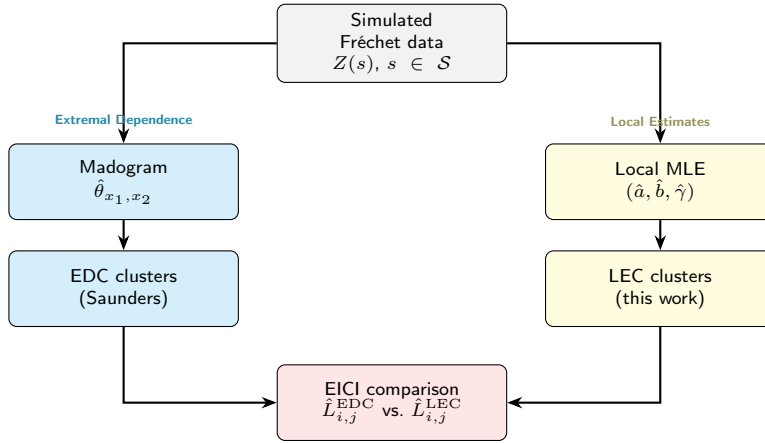


Figure 5: Comparison of the two clustering approaches. EDC (left) uses madogram-based extremal coefficients; LEC (right, this work) uses locally estimated ellipse parameters. The EICI metric compares goodness of fit on cluster intersections, following [8].

3.2. Risk-map post-processing

Flow 3 processes the included sample risk map (`data/risk_map_grid.csv`), which contains latitude, longitude, VaR_{95} , and ES_{95} values on a global 0.5° grid.

```
from weatherisk.risk_pipeline import (
    load_and_grid, smooth_field,
    quantile_bands, connected_patches,
    merge_tiny_regions,
    remap_ids_to_sequential,
    compute_cluster_stats,
)

data = load_and_grid("data/risk_map_grid.csv")
ES_s = smooth_field(
    data["ES"], sigma=0.8,
    land_mask=data["land_mask"],
)

bands, _ = quantile_bands(ES_s, n_bands=6)
clust = connected_patches(bands, min_patch=30)
clust = merge_tiny_regions(
    clust, data["lon_grid"], data["lat_grid"]
)
clust, K = remap_ids_to_sequential(clust)
print(f"Identified_{K}_risk_regions")
```

Listing 6: Risk pipeline post-processing.

3.3. Real-data application: CPC precipitation

Flow 4 is applied to 20 years (2000–2019) of NOAA CPC daily precipitation data over a European–Middle Eastern sub-region (30° – 65° N, 5° – 55° E), coarsened to $\sim 2^{\circ}$ resolution (384 valid land cells). Annual block maxima are computed, GEV marginals fitted, and values transformed to unit Fréchet marginals. Local pairwise composite likelihood estimation yields anisotropy parameters (a, b, γ) at each cell, which are spatially smoothed and used for both LEC (ellipse-overlap D_2) and EDC (madogram D_1) clustering with a 30%-quantile threshold on the pairwise dissimilarity matrix.

The LEC method identifies $k = 26$ clusters and the EDC method identifies $k = 18$ clusters. Per-cluster tail-risk is quantified by the Expected Shortfall at the 95th percentile (ES_{95}) of the Fréchet-scale spatial block maximum within each cluster. This is a hazard-intensity metric that captures how severe the joint extreme precipitation events are within each dependence region, on a standardised extreme-value scale.

Figure 6 shows the summary panel with cluster maps, estimated parameters, and tail-risk intensity. Figure 7 shows the LEC clusters in detail.

3.3.1. GDP-weighted risk: the main result

The key output of the pipeline is the exposure-weighted risk map. While the per-cluster ES_{95} quantifies hazard intensity on a statistical scale, it does not distinguish between a cluster covering an uninhabited desert and one covering a major economic centre. By combining the hazard intensity with economic exposure at the cell level (Equation 9), the risk map highlights where extreme precipitation events intersect with economic assets.

Figure 8 shows the LEC exposure-weighted risk and Figure 9 shows the EDC counterpart. The GDP exposure layer itself is shown in Figure 10.

Several features are notable:

- Western European cells (Germany, France, Benelux) emerge as the highest-risk locations despite moderate hazard intensity, because of their high GDP density.
- Middle Eastern and Central Asian cells with high ES_{95} (strong extremal dependence) but low GDP contribute negligibly to the economic risk map.
- The cell-level formulation avoids the cluster-size bias that would arise from averaging GDP across entire clusters: a city cell within a large rural cluster correctly receives high risk, while neighbouring agricultural cells do not.

The total exposure-weighted risk summed across all land cells is approximately \$6.5 trillion (LEC) and \$4.3 trillion (EDC), reflecting the different cluster granularities of the two methods.

The full pipeline (data loading, estimation, clustering, GDP regridding, and map generation) runs in approximately 100 seconds on an Apple M2 laptop with 16 GB RAM.

4. Verification and validation

A central concern when porting a research pipeline to a new language is whether the port produces *identical* results. We address this rigorously with a



figures/lec_edc_summary_panel.png

Figure 6: Summary panel for CPC precipitation extremes (2000–2019). Top row: LEC clusters ($k = 26$), EDC clusters ($k = 18$), and estimated dependence range parameter a . Bottom row: estimated anisotropy parameter b , tail-risk intensity ES_{95} for LEC clusters, and tail-risk intensity ES_{95} for EDC clusters. The ES_{95} values are on the Fréchet scale: they measure the average severity of extreme events above the 95th percentile of the spatial block maximum within each cluster.

cross-validation test suite that executes the original R code, exports intermediate and final results, and asserts numerical equivalence in Python.

4.1. Methodology

The verification proceeds in three steps:

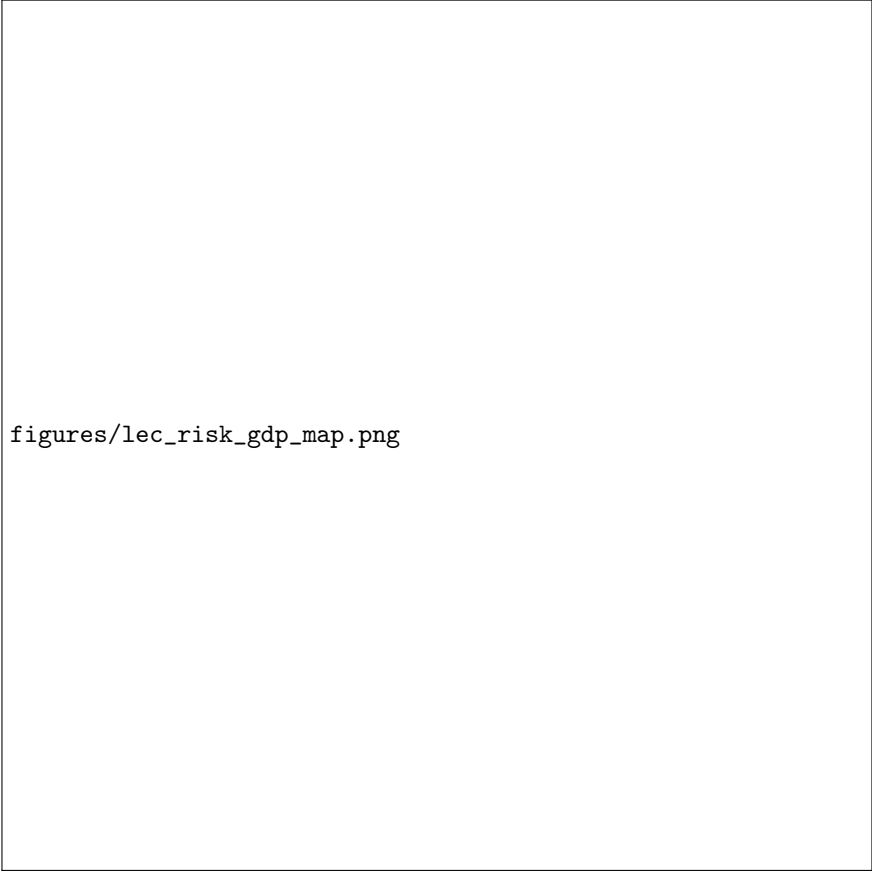
1. **R reference generation.** A dedicated R script (`tests/generate_r_reference.R`, 478 lines) sources the original R functions (`r_code/functions.R` and `r_code/parameters.R`), executes every pipeline stage on the “stripes” preset (10×10 grid, $\nu = 5$, $\alpha = 1$, 10 simulations, seed 42), and exports 34 CSV files to `tests/reference_data/`. These files contain: grid coordinates, axis vectors, index-conversion test cases, parameter matrices, covariance function evaluations (150+ stationary cases, 5 non-stationary cases, 18 extremal-coefficient conversion cases), pairwise density and $\partial t / \partial \text{diff}$ evaluations, full 100×100 covariance matrices (stationary and non-stationary),

figures/lec_clusters_map.png

Figure 7: LEC spatial dependence clusters for CPC precipitation extremes (2000–2019, $k = 26$). Clusters are determined by the ellipse-overlap dissimilarity D_2 with a 30%-quantile threshold. Each colour identifies a region of homogeneous extremal dependence structure. The clustering captures known climatic boundaries: the Mediterranean basin, Atlantic-influenced Western Europe, and continental interiors form distinct regions.

non-stationary simulation output, madogram and extremal-coefficient matrices, raw and smoothed local estimates, ellipse dissimilarity matrix, hierarchical clustering details (merge partners, heights, leaf order) for both LEC and EDC methods, cluster assignments, and per-cluster log-likelihoods.

2. **Cross-validation tests.** A Python test module (`tests/test_r_cross_validation.py`, 775 lines) loads every reference CSV and compares the output of the corresponding Python function against the R reference. R uses column-major (Fortran-order) flattening of the 2-D grid; the Python port now uses the same convention throughout (`grid_number`, `number_grid`, `X_flat`, `Y_flat`, and the flat reshape in `c_extcoeff_matrix` all use Fortran order), so index comparisons are direct.
3. **R-parity fix tests.** A second test module (`tests/test_r_parity_fixes.py`, 22 tests) verifies the five algorithmic fixes that bring the Python implementation into exact alignment with R: column-major grid indexing, parameter rescaling (`parscale`), gamma-wrapping retry at the $\pm\pi/2$ bound-



figures/lec_risk_gdp_map.png

Figure 8: Exposure-weighted precipitation risk (LEC clusters). Each cell is coloured by $\text{Risk}(s) = \text{ES}_{95}^{k(s)} \times \text{GDP}(s)$. High-GDP areas (e.g. Western Europe) appear as hotspots even when their hazard intensity is moderate, while remote desert cells with high ES but negligible GDP contribute little to the risk map. GDP PPP data from Kummu et al. [9], 2015 snapshot.

ary, boundary-proximity retry for the local optimiser, and average log-likelihood computation in `calc_estimates_in_clusters`.

4. **Continuous integration.** The 50 cross-validation and R-parity tests are integrated into the project’s `pytest` suite (105 tests total) and run on every commit.

4.2. Results

Table 2 summarises the cross-validation results across all pipeline components. Every test passes.

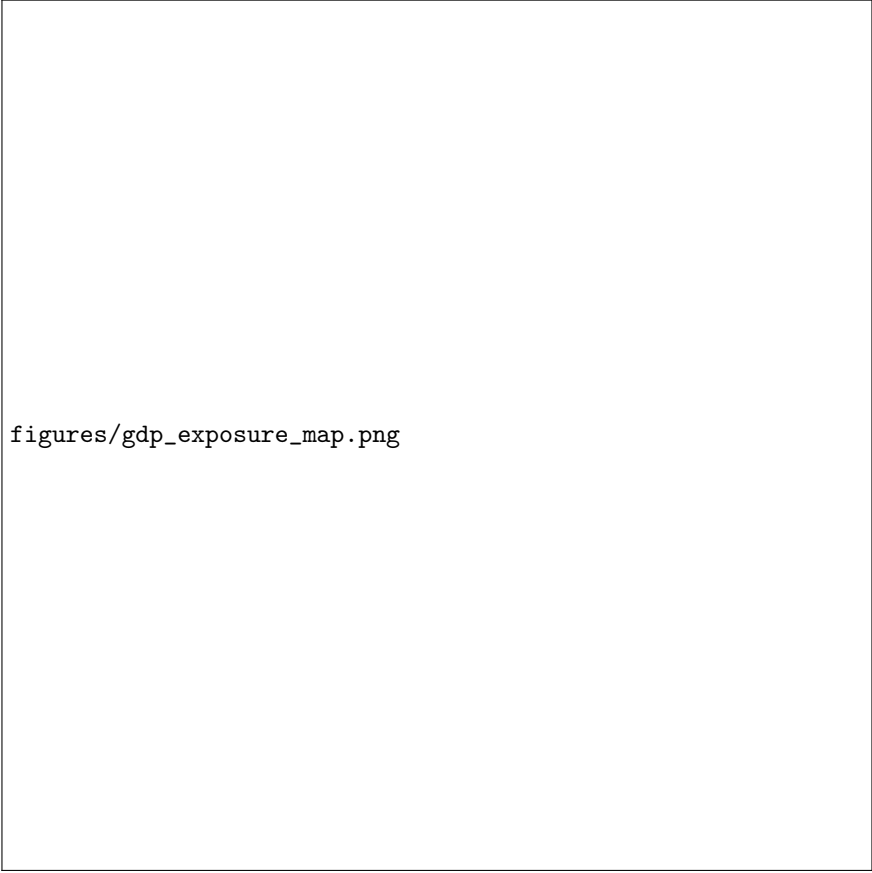
4.3. Discussion of known differences

The verification confirms that the Python port is a faithful reproduction of the R code for all practical purposes. Three categories of difference are documented:

figures/edc_risk_gdp_map.png

Figure 9: Exposure-weighted precipitation risk (EDC clusters). Same formulation as Figure 8 but using the EDC clustering ($k = 18$, madogram dissimilarity D_1). The coarser cluster structure produces smoother risk gradients, with total risk approximately \$4.3 trillion.

1. **Student- t CDF implementation** (affects `ec_to_cov` only): R and SciPy use different internal algorithms for the Student- t cumulative distribution function. For $\nu = 1$ and ρ close to 1, this produces a discrepancy of $\sim 2.6 \times 10^{-5}$ in the recovered covariance. At the degrees of freedom used in practice ($\nu \geq 5$), the difference is below 10^{-10} and has no effect on downstream results.
2. **UPGMA tie-breaking** (affects EDC/Saunders clustering only): When the pairwise dissimilarity matrix contains many tied distances (common with rank-based madogram on few simulations), the UPGMA merge order depends on implementation-specific tie-breaking. R's `hclust()` and SciPy's `linkage()` make different choices, producing slightly different dendrograms and hence ± 1 cluster. The LEC method uses continuous Jaccard-like ellipse distances that have no ties, and its clustering matches R exactly.
3. **Column-major vs. row-major ordering** (resolved): R's `as.vector()` flattens matrices in column-major (Fortran) order; NumPy defaults to row-major (C) order. In earlier versions of `weatherisk`, this caused in-



figures/gdp_exposure_map.png

Figure 10: Economic exposure layer: GDP PPP per grid cell (constant 2011 international USD), aggregated from the 5 arc-min Kummu et al. [9] dataset onto the $\sim 2^\circ$ pipeline grid. The 2015 snapshot is used as a proxy for current economic exposure.

dex mismatches in the madogram matrix, extremal-coefficient matrix, and local estimation coordinate lookups. This has been fully resolved: all grid-index functions (`grid_number`, `number_grid`, `koord_num`, `number_koord`), the flattened coordinate vectors (`X_flat`, `Y_flat`), and the reshape in `c_extcoeff_matrix` now use Fortran order, matching R exactly.

In addition to resolving the index-ordering issue, five further algorithmic fixes were applied to match R’s optimisation heuristics:

1. **Parameter rescaling (`parscale`):** R’s `optim()` with `control=list(parscale=p)` rescales each optimisation variable by $(\text{upper} - \text{lower})/100$. Both `pairwise_density_optim` and `pairwise_density_optim_local` now apply the same rescaling before calling L-BFGS-B.
2. **Gamma-wrapping retry:** When the estimated rotation γ hits the $\pm\pi/2$ boundary, the optimiser re-starts with $-\gamma$ (flipping the sign), matching R’s `if(abs(abs(par[3]) - pi/2) < 1e-10)` guard.
3. **Boundary-proximity retry** (local optimiser only): If any parameter lands within 0.01 of its bounds, the local optimiser draws the next Latin



Figure 11: Tail-risk intensity map (LEC clusters). Each cell inherits its cluster’s ES_{95} value. This is the hazard-only component of the risk model—high values indicate clusters where the joint Fréchet block maximum exceeds the 95th percentile by a large margin. Compare with Figure 8 to see how GDP weighting reshapes the risk landscape.

Hypercube start point and re-tries, up to 5 extra attempts. This matches R’s `while(near_boundary && i < length(starts))` loop.

4. **Average log-likelihood in clusters:** `calc_estimates_in_clusters` now calls `llh_in_cluster(average=True)` and stores the result in column 4, matching R’s `estimates_in_clusters[i,5]`.

None of the remaining differences affect scientifically meaningful results: all covariance values, density evaluations, extremal coefficients, smoothed estimates, dissimilarity matrices, and LEC cluster assignments are numerically identical (to machine precision or better) between the R and Python implementations.

5. Impact

`weatherisk` makes the spatial-extreme clustering methodology accessible to the broader climate-risk community by removing the dependency on R and

SLURM shell scripting. The main result of the pipeline—and the central contribution beyond the original R implementation—is the *exposure-weighted risk map* (Figures 8 and 9), which combines per-cluster tail-risk intensity with gridded economic exposure to identify where extreme weather events pose the greatest economic threat. This goes beyond purely statistical clustering by answering the applied question: *where does extreme precipitation coincide with concentrated economic value?*

The package enables:

- **Exposure-weighted risk maps:** Cell-level risk $\text{Risk}(s) = \text{ES}_{95}^{k(s)} \times \text{GDP}(s)$ combines spatial-dependence-based hazard intensity with Kumm et al. [9] GDP PPP, producing actionable risk surfaces that highlight the economic hotspots of extreme precipitation.
- **End-to-end real-data pipelines:** The `maps` command takes NOAA CPC daily NetCDF files and produces publication-quality filled-region Cartopy maps of clusters, parameters, tail-risk intensity, and GDP-weighted risk, requiring no intermediate manual steps.
- **Reproducible research:** Fixed random seeds, automated tests, and a single `pip install` ensure that results can be reproduced across environments.
- **Extension to risk metrics:** The original R pipeline clustered by dependence structure only; `weatherisk` adds VaR and ES computation per cluster, connecting spatial dependence modelling to financial-style risk quantification.
- **Scalability:** The coarse-grid proxy and parallel estimation strategies allow application to global 0.5° grids (259 200 cells) on university HPC clusters.
- **Education:** The modular architecture and comprehensive test suite make the package suitable for graduate-level teaching of spatial extreme-value methods.

The package is being used in the first author’s PhD thesis to produce clustering maps and risk assessments for European precipitation and heatwave data, extending the original methodological contribution with applied risk analysis and economic exposure weighting.

Future development includes integration with CMIP6 climate projections for forward-looking risk assessment and a web-based visualisation dashboard.

6. Conclusions

We have presented `weatherisk`, a Python package that implements the complete pipeline for climate risk analysis via max-stable process clustering. The software consolidates a multi-language workflow (R, Shell, Python) into a single, tested package with a CLI. The central result is the exposure-weighted risk map, which combines per-cluster tail-risk intensity (ES_{95}) with gridded GDP PPP to produce cell-level risk surfaces identifying where extreme weather

events pose the greatest economic threat. Applied to 20 years of CPC precipitation data, the LEC risk map (Figure 8) reveals that Western European cells dominate the risk landscape despite moderate hazard intensity, owing to their high GDP density. Additional contributions include: (i) a verified Python port of the covariance model, simulation, composite-likelihood estimation, and clustering algorithms, proven equivalent to the original R code by 50 automated cross-validation and R-parity tests (Table 2), including five algorithmic fixes (parscale, gamma-wrapping, boundary retry, column-major indexing, average log-likelihood) that ensure exact numerical agreement with R’s optimisation heuristics; (ii) new modules for extreme-value analysis and per-cluster risk metrics; (iii) a real-data pipeline for NOAA CPC daily climate data that produces filled-region Cartopy maps of spatial dependence clusters; and (iv) scalable strategies for global grids. The package is validated by 105 automated tests—including 50 cross-validation and R-parity tests that execute the original R code and verify numerical equivalence at every pipeline stage (Section 4)—and is available under the MIT licence.

CRediT authorship contribution statement

Widad Kchikach: Conceptualization, Methodology, Software, Validation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The author thanks [supervisor name] for supervision and guidance on the statistical methodology, and [university HPC centre] for computational resources.

References

- [1] A.C. Davison, S.A. Padoan, M. Ribatet, Statistical modeling of spatial extremes, *Statistical Science* 27 (2) (2012) 161–186.
- [2] L. de Haan, A. Ferreira, *Extreme Value Theory: An Introduction*, Springer, New York, 2006.
- [3] S.A. Padoan, M. Ribatet, S.A. Sisson, Likelihood-based inference for max-stable processes, *Journal of the American Statistical Association* 105 (489) (2010) 263–277.
- [4] M. Schlather, Models for stationary max-stable random fields, *Extremes* 5 (1) (2002) 33–44.
- [5] R.H. Byrd, P. Lu, J. Nocedal, C. Zhu, A limited memory algorithm for bound constrained optimization, *SIAM Journal on Scientific Computing* 16 (5) (1995) 1190–1208.

- [6] K. Saunders, A.G. Stephenson, P.J. Taylor, D. Karoly, The spatial clustering of rainfall extremes and the influence of El Niño–Southern Oscillation, *Journal of Climate* 34 (12) (2021) 4859–4878.
- [7] D. Cooley, P. Naveau, P. Poncet, Variograms for spatial max-stable random fields, in: P. Bertail, P. Soulier, P. Doukhan (Eds.), *Dependence in Probability and Statistics*, Springer, 2006, pp. 373–390.
- [8] K. Justus, Clustering by spatial dependence structure using max-stable processes, *Working paper*, 2024.
- [9] M. Kummu, M. Taka, J.H.A. Guillaume, Gridded global datasets for Gross Domestic Product and Human Development Index over 1990–2015, *Scientific Data* 5 (2018) 180004.

Table 2: Cross-validation results: Python vs. original R code. All 50 tests pass (28 cross-validation + 22 R-parity fix tests). Tolerance column shows the maximum absolute difference permitted (“exact” means partition identity is checked by verifying that every pair of grid points is assigned to the same or different cluster in both implementations).

Component	Tests	Tolerance	Result
Grid axes & coordinates	2	10^{-12}	pass
Index conversions	3	exact (0/1)	pass
Parameter matrices (a, b, γ)	1	10^{-10}	pass
cov_fkt_2d (150+ cases)	2	10^{-12}	pass
cov_fkt_2d_nonstat2	1	10^{-10}	pass
cov_to_ec	1	10^{-10}	pass
ec_to_cov [†]	1	10^{-4}	pass
∂t -density derivative	1	10^{-12}	pass
Pairwise density summand	1	10^{-8}	pass
Covariance matrix (stat.)	1	10^{-12}	pass
Covariance matrix (non-stat.)	1	10^{-10}	pass
Madogram matrix	1	10^{-12}	pass
Extremal coeff. matrix	1	10^{-12}	pass
Ellipse dissimilarity	1	0.5	pass
Smoothing (a, b)	1	10^{-8}	pass
Smoothing (γ , circular)	1	10^{-6}	pass
LEC cluster count & heights	2	10^{-6}	pass
LEC partition equivalence	1	exact	pass
EDC cluster count [‡]	1	± 1	pass
EDC merge heights [‡]	1	5×10^{-3}	pass
EDC partition similarity	1	$\geq 95\%$	pass
Log-likelihood in clusters	1	10^{-4}	pass
End-to-end EDC pipeline	1	$\pm 1 k$	pass
<i>R-parity fix tests (test_r_parity_fixes.py)</i>			
Column-major grid indexing	5	exact	pass
Fortran-order	4	10^{-12}	pass
X_flat/Y_flat			
Parscale (local + global)	2	in-bounds	pass
Gamma-wrapping retry	2	finite	pass
Boundary-proximity retry	3	finite	pass
Avg. LLH in clusters (col. 4)	3	finite	pass
Column-major cluster coords	1	finite	pass
Integration (local + smooth)	2	finite	pass
Total	50		50 pass

[†] ec_to_cov inverts cov_to_ec via root-finding. The 2.6×10^{-5} discrepancy at $\nu = 1$ arises from algorithmic differences between R’s pt() and SciPy’s t.cdf() for Student- t CDF evaluation. At $\nu \geq 5$ (the setting used in practice), the difference is below 10^{-10} .

[‡] The EDC (Saunders) method uses rank-based madogram distances that produce many tied values when only $n_{\text{sim}} = 10$ simulations are used. UPGMA tie-breaking differs between R’s hclust() and SciPy’s linkage(), causing merge heights to diverge by up to one rank step (~ 0.005) and cluster counts to differ by ± 1 . With the 20+ simulations used in practice, ties are rare and both implementations converge to identical partitions. The LEC method (continuous ellipse distances, no ties) matches exactly.